Exercise 2 - Documentation, errors and user input

In this exercise you will practise what we saw in the second session:

- **Part 1** – comments and docstrings
- **Part 2** – handling errors with `try` and `except`
- **Part 3** – asking the user for input

Beginners: Use the hints in `Exercise 2 – step by step – day 4` whenever you get stuck, and please ask if something is unclear.

Advanced: Try to solve everything without the hints.

Part 1 - Comments and docstrings

A colleague wrote this function and gave it to you. Run the cell.

In [ ]:
```
def f(s):
    # count the G's
    n = 0
    for x in s:
        if x == 'A' or x == 'T':
            n += 1
    return n / len(s) * 100
```

In [ ]:
```
f('ATGCAT')
```

1. Read the code. What does the function **really** calculate? Is the comment correct?
2. Rewrite the function:
   - give it a better name, `at_content`, and call the parameter `sequence`
   - fix (or remove) the wrong comment
   - add a **docstring** that explains what the function does, its parameter, and what it returns
3. Check your docstring with `help(at_content)`.

In [ ]:
```

```

In [ ]:
```

```

Part 2 - Handling errors

A sequencing machine wrote the length of each read it produced. Unfortunately, some values are broken:

In [ ]:
```
read_lengths = ['150', '148', '151', 'N/A', '150', 'error', '149']
```

We want the **average read length** of the valid reads.

1. Loop over `read_lengths` and convert each value to an integer with `int()`. Run it. What happens? Look at the **last line** of the error message: which type of error is it?

In [ ]:
```

```

2. Use `try` and `except` so that the broken values are **skipped**. For each skipped value, print a message like:
   `Skipping value: N/A`
3. Calculate and print the average length of the valid reads. You should get:
   `Average read length: 149.6`

In [ ]:
```

```

Part 3 - User input

Now let the **user** decide which reads are long enough.

1. Ask the user for a minimum read length with `input()`. What is the **type** of the value you get?
2. Convert it to an integer. If the user does not give an integer (for example `abc` or `149.5`), print a message and **ask again**, until you get an integer.
3. Print all valid reads from Part 2 that are at least as long as the minimum. For minimum `150` you should get:

```
Give a minimum read length: abc
'abc' is not an integer. Try again!
Give a minimum read length: 150
Reads with length >= 150: [150, 151, 150]
```

In [ ]:
```

```

In [ ]:
```

```